

Week 7 - Image Segmentation (Clustering, Region-Based, and Edge-Based Methods)

[28/Mar/2026 Dr Min Wang]

Overview

Image segmentation partitions an image into meaningful regions corresponding to objects or structures. In this lab, you will explore three fundamental image segmentation techniques:

1. K-Means Clustering (Similarity-based)
2. Watershed Algorithm (Region-based)
3. Edge-based Segmentation (Canny + Contours)

Objectives

- Understand different segmentation principles
- Implement and compare three methods
- Analyze how parameters affect results

Requirements

You need the following libraries, you should have been familiar with them by now, but install them if you haven't done so.

pip install numpy opencv-python matplotlib scikit-learn

Load example image and visualise

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Load image
image = cv2.imread('horse.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert to Grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(8,4))

plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(gray, cmap='gray')
plt.title("Grayscale Image")
plt.axis('off')

plt.show()
```

Original Image



Grayscale Image



Image Segmentation

Part A: K-Means Segmentation (Similarity-Based)

Concept: Clusters pixels based on color similarity in feature space (e.g., RGB).

- First we need to prepare data (reshape image into pixel list) for K-means algorithm
- Apply K-Means
- Then we can reconstruct the segmented image

```
In [2]: # Reshape into pixel list
pixel_values = image_rgb.reshape((-1, 3))
pixel_values = np.float32(pixel_values)

print("Shape of pixel array:", pixel_values.shape)
```

Shape of pixel array: (96000, 3)

```
In [3]: # Apply K-Means
k = 2
kmeans = KMeans(n_clusters=k, random_state=0)
labels = kmeans.fit_predict(pixel_values)
centers = np.uint8(kmeans.cluster_centers_)

print("Cluster centers:\n", centers)
```

Cluster centers:

```
[[201 199 198]
 [ 30  37  36]]
```

```
In [4]: # Reconstruct segmented image
segmented_image = centers[labels.flatten()]
segmented_image = segmented_image.reshape(image_rgb.shape)

# Display
plt.figure(figsize=(8,4))
plt.subplot(1,2,1)
plt.title("Original")
plt.imshow(image_rgb)
plt.axis('off')

plt.subplot(1,2,2)
plt.title(f"K-Means Segmentation (k={k})")
plt.imshow(segmented_image)
plt.axis('off')
plt.show()
```

Original



K-Means Segmentation (k=2)



Discussion

In this exmple, each pixel is treated as a point in RGB space. K-Means groups pixels into k clusters. Each cluster is replaced by its centroid color.

Next, try the following and discuss your findings with the tutor:

- Try k = 2, 3, 4, 5.
- Convert image to grayscale before clustering.
- Apply to different image types.

Part B: Watershed Segmentation (Region-Based)

Concept: The Watershed algorithm treats the image as a topographic surface:

- Bright pixels → peaks
- Dark pixels → valleys
- Flooding from markers forms regions.

```
In [5]: # Preprocessing: remove small fluctuations (noise)
blur = cv2.GaussianBlur(gray, (3,3), 0)

# Thresholding (separate foreground and background),
# this converts image to binary and gives a rough idea of where objects are (initial guess)
_, thresh = cv2.threshold(blur, 0, 255,
                          cv2.THRESH_BINARY + cv2.THRESH_OTSU)

plt.figure(figsize=(4,4))
plt.imshow(thresh, cmap='gray')
plt.title("Thresholded Image")
plt.axis('off')
plt.show()
```

Thresholded Image

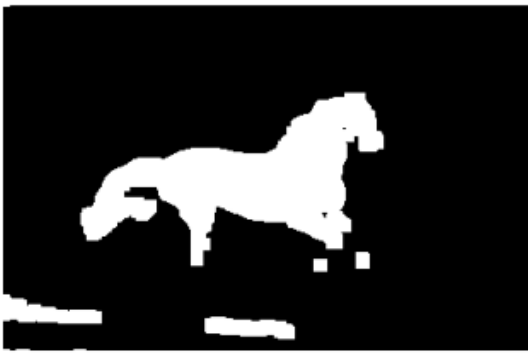


```
In [6]: # Morphological Opening (this removes small white noise in the mask)
kernel = np.ones((3,3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)

# Sure background
sure_bg = cv2.dilate(opening, kernel, iterations=2)

plt.figure(figsize=(4,4))
plt.imshow(sure_bg, cmap='gray')
plt.title("Sure Background")
plt.axis('off')
plt.show()
```

Sure Background

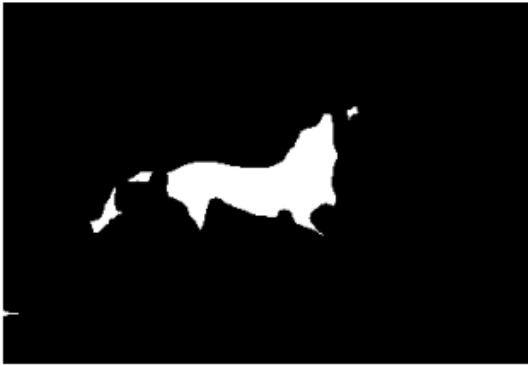


```
In [7]: # Distance transform (find object centers)
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)

# Sure foreground (keeps only the central parts of objects, high-confidence regions)
_, sure_fg = cv2.threshold(dist_transform,
                           0.2*dist_transform.max(), 255, 0)

plt.figure(figsize=(4,4))
plt.imshow(sure_fg, cmap='gray')
plt.title("Sure Foreground")
plt.axis('off')
plt.show()
```

Sure Foreground



```
In [8]: # Unknown region
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

plt.figure(figsize=(4,4))
plt.imshow(unknown, cmap='gray')
plt.title("Unknown Region")
plt.axis('off')
plt.show()
```

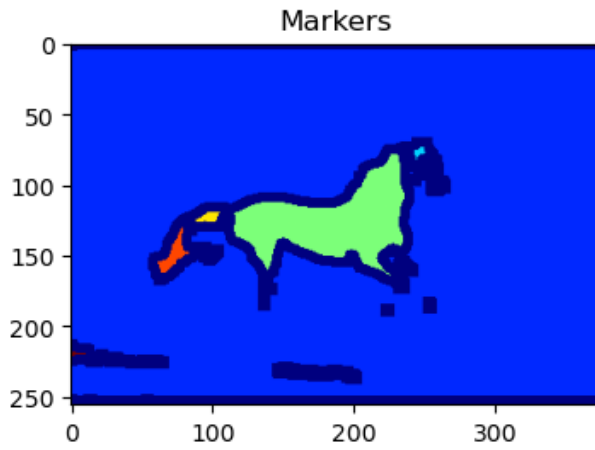
Unknown Region



```
In [9]: # Markers
_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

plt.figure(figsize=(4,4))
plt.imshow(markers, cmap='jet')
```

```
plt.title("Markers")
#plt.colorbar()
plt.show()
```



```
In [10]: # Apply Watershed (floods from markers, regions expand until they meet)
image_ws = image.copy()

markers = cv2.watershed(image_ws, markers)
image_ws[markers == -1] = [255, 0, 0]

# Display
plt.figure(figsize=(8,4))

plt.subplot(1,2,1)
plt.title("Original")
plt.imshow(image_rgb)
plt.axis('off')

plt.subplot(1,2,2)
plt.title("Watershed Segmentation")
plt.imshow(image_ws)
plt.axis('off')

plt.show()
```

Original



Watershed Segmentation



Discussion

In this example, we follow four steps:

- Thresholding separates foreground/background.
- Distance transform finds object centers.
- Markers guide region growing.
- Watershed assigns pixels to regions until boundaries meet.

Try the following and discuss the findings with your tutor:

- Change threshold factor (e.g., 0.2, 0.5, 0.8).
- Modify kernel size.

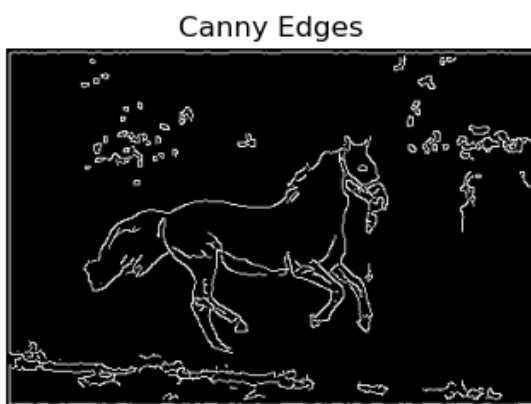
Part C: Edge-Based Segmentation (Canny + Contours)

Concept: here we first detect object boundaries using intensity discontinuities (edges).

```
In [11]: # Smooth image
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Edge detection
edges = cv2.Canny(blur, 50, 150)

plt.figure(figsize=(4,4))
plt.imshow(edges, cmap='gray')
plt.title("Canny Edges")
plt.axis('off')
plt.show()
```



```
In [12]: # Find contours
contours, _ = cv2.findContours(
    edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

print("Number of contours:", len(contours))
```

Number of contours: 98

```
In [13]: # Draw contours
contour_image = image_rgb.copy()
cv2.drawContours(contour_image, contours, -1, (255, 0, 0), 1)

# Display results
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.title("Original")
plt.imshow(image_rgb)
plt.axis('off')

plt.subplot(1,3,2)
plt.title("Edges")
plt.imshow(edges, cmap='gray')
plt.axis('off')

plt.subplot(1,3,3)
plt.title("Contours")
plt.imshow(contour_image)
plt.axis('off')

plt.show()
```

Original



Edges



Contours



Discussion

In the example, we

- Apply Canny to detect edges via gradients.
- Then contour group connected edge pixels.
- And finally produce boundary-based segmentation.

Try yourself and discuss your observation with your tutor:

- Tune thresholds (e.g., 30–100, 100–200).

Summary

Aspect	K-Means	Watershed	Edge-Based
Principle	Similarity	Region growing	Discontinuity
Spatial info	No	Yes	Partial
Handles overlap	No	Yes	No
Noise sensitivity	Medium	High	High
Output	Region labels	Regions + boundaries	Boundaries

Discussion Questions

- How does changing K affect segmentation quality?
- Why does K-Means ignore spatial relationships?
- Why is watershed sensitive to noise and initialization?
- What role does the distance transform play?
- Why do edge-based methods often produce broken boundaries?
- Which method works best for:
 - medical images?
 - natural scenes?
 - industrial inspection?
- How could you combine these methods for better results?

In []: